Fine-tuning Language Models for Code Translation and Generation: A Comparative Study of Full Parameter and LoRA Approaches

Shujun Liao
ge83juz@tum.de
Technical University of Munich
Heilbronn, Baden-Württemberg, Germany

Vipin Chaudhry
go98vib@mytum.de
Technical University of Munich
Heilbronn, Baden-Württemberg, Germany

***Abstract*—This paper presents a comprehensive study on fine-tuning large language models for code-related tasks, specifically focusing on cross-language code translation and code generation. We investigate two distinct fine-tuning approaches: full parameter fine-tuning and Low-Rank Adaptation (LoRA). For code translation, we fine-tune CodeT5 on the XLCoST dataset for Java to C# translation, achieving a BLEU score improvement from 54.17 to 100.00, CodeBLEU from 0.1029 to 0.9390, and a functional correctness rate of 72.06%. For code generation, we apply LoRA fine-tuning to deepseek-coder-6.7b-instruct and Qwen2.5-Coder-3B-Instruct models for Python code synthesis from natural language descriptions. Our experiments demonstrate that LoRA enables efficient fine-tuning with only 1-2% trainable parameters while achieving competitive performance. DeepSeek Coder shows significant improvement with Pass@1 increasing from 39.70% to 57.00%, while Qwen achieves even larger gains from 33.55% to 61.20%. Both models show dramatic reductions in syntax and runtime errors after LoRA fine-tuning. Our findings provide insights into the trade-offs between full fine-tuning and parameter-efficient methods for code intelligence tasks.**

***Index Terms*—Code Translation, Code Generation, Large Language Models, Fine-tuning, LoRA, CodeT5, DeepSeek Coder, Qwen Coder**

I. Introduction

The emergence of large language models (LLMs) has revolutionized the field of software engineering, enabling automated code generation, translation, and analysis at unprecedented levels of quality [8]. As software systems become increasingly complex and multilingual, the ability to automatically translate code between programming languages and generate functional code from natural language specifications has become crucial for developer productivity and software maintenance.

Code translation, the task of converting source code from one programming language to another while preserving functionality, addresses critical challenges in software migration and modernization. Organizations frequently need to migrate legacy systems from older languages to modern platforms, a process that is traditionally manual, error-prone, and expensive. The Java to C# translation task is particularly relevant given the widespread adoption of both languages in enterprise environments and their syntactic similarities that make neural translation feasible.

Similarly, code generation from natural language descriptions has the potential to democratize software development by allowing developers to express their intent in plain language rather than detailed syntax. This capability is especially valuable for rapid prototyping, educational purposes, and assisting developers with unfamiliar APIs or algorithms.

Pre-trained code models such as CodeT5 [1], CodeBERT [2], and more recent decoder-only models like DeepSeek Coder [3] and Qwen2.5 Coder [14] have shown impressive capabilities on various code understanding and generation tasks. However, these models typically require task-specific fine-tuning to achieve optimal performance on downstream applications.

Fine-tuning approaches can be broadly categorized into two paradigms: full parameter fine-tuning and parameter-efficient fine-tuning (PEFT). Full fine-tuning updates all model parameters and often achieves the best task-specific performance but requires significant computational resources. In contrast, PEFT methods such as Low-Rank Adaptation (LoRA) [4] update only a small fraction of parameters while maintaining competitive performance, making them attractive for resource-constrained environments.

In this paper, we present a comparative study of these two fine-tuning approaches applied to code translation and code generation tasks. Our main contributions are:

- We demonstrate that full fine-tuning of CodeT5 (222.88M parameters) on the XLCoST dataset achieves state-of-the-art performance on Java to C# translation, with BLEU scores improving by 84.6%, CodeBLEU by 813%, and functional correctness reaching 72.06%.
- We evaluate LoRA fine-tuning on two modern code LLMs (deepseek-coder-6.7b-instruct and Qwen2.5-Coder-3B-Instruct) for Python code generation, showing that training only 1.13-1.73% of parameters can yield significant improvements in Pass@1 metrics and dramatically reduce syntax errors.
- We provide a comprehensive analysis comparing full fine-tuning and LoRA approaches in terms of training

efficiency, resource requirements, error distribution, and downstream task performance.
- We present detailed error analysis and code quality metrics, illustrating common failure patterns and the impact of fine-tuning on code style and correctness.

II. Related Work

A. Neural Code Translation

Neural machine translation techniques have been successfully adapted for code translation tasks. TransCoder [5] pioneered unsupervised code translation using denoising auto-encoding and back-translation, demonstrating that large-scale pre-training on monolingual code corpora enables effective cross-language translation without parallel data. PLBART [6] extended BART to programming languages, achieving strong results on code translation benchmarks through unified pre-training on both natural language and code.

CodeT5 [1] introduced identifier-aware pre-training objectives that help the model understand the semantic meaning of code tokens, particularly variable and function names. This approach proved especially effective for code translation tasks where preserving semantic relationships is crucial.

The XLCoST benchmark [7] provides parallel code corpora across seven programming languages (C++, Java, Python, C#, JavaScript, PHP, and C), enabling supervised training and evaluation of code translation models at both snippet and program levels.

B. Code Generation from Natural Language

Generating executable code from natural language descriptions has been a long-standing goal in software engineering research. Early approaches relied on semantic parsing and domain-specific languages, which were limited in scope and flexibility.

The advent of transformer-based models brought significant improvements, with models like Codex [8] demonstrating remarkable code generation capabilities. GPT-4 and subsequent models have achieved even higher performance on code benchmarks, though they remain proprietary.

Recent open-source models such as StarCoder [9], DeepSeek Coder [3], and Qwen Coder [14] have achieved competitive performance with proprietary models. DeepSeek Coder is trained on 2 trillion tokens from 87 programming languages and demonstrates strong instruction-following capabilities. These models are pre-trained on large-scale code corpora and can be further fine-tuned for specific tasks.

The HumanEval benchmark [8] introduced the Pass@k metric for evaluating functional correctness of generated code, which has become the standard evaluation protocol. This metric measures the probability that at least one of k generated samples passes all unit tests.

C. Parameter-Efficient Fine-Tuning

As language models grow in size, full fine-tuning becomes increasingly expensive in terms of both computation and memory. Parameter-efficient fine-tuning (PEFT) methods address this challenge by updating only a small subset of model parameters while keeping the majority frozen.

Adapter layers [10] insert small trainable modules between transformer layers, typically adding 1-5% additional parameters. Prefix tuning [11] prepends learnable tokens to the input, modifying the model's attention patterns without changing its weights.

Low-Rank Adaptation (LoRA) [4] represents weight updates as low-rank decomposition matrices, enabling efficient fine-tuning with minimal additional parameters. For a weight matrix $W_0 \in \mathbb{R}^{d\times k}$, LoRA adds $\Delta W = BA$ where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ with rank $r \ll \min(d, k)$.

QLoRA [12] combines LoRA with 4-bit quantization, allowing fine-tuning of models with billions of parameters on consumer-grade GPUs with as little as 8GB VRAM. This democratizes access to fine-tuning large models for academic and individual researchers.

III. Methodology

This section describes our two-part approach to code intelligence tasks: full parameter fine-tuning for code translation and LoRA-based fine-tuning for code generation.

A. Implementation Pipeline

We developed two integrated training pipelines for our experiments: **Kabul** for code translation and **LLMTrainPipeline** for code generation.

1) Kabul: Code Translation Pipeline: The Kabul pipeline implements a modular architecture for training and evaluating code translation models:

- **Training Module** (`train_model.py`): Orchestrates the fine-tuning process using HuggingFace's Seq2SeqTrainer with custom callbacks for detailed logging and progress reporting.
- **Evaluation Module** (`evaluate_model.py`): Provides multidimensional evaluation including BLEU, CodeBLEU (with AST and DataFlow analysis), compilation testing via .NET SDK, and functional correctness verification.
- **Post-Processing Pipeline** (`post_process.py`): Implements a three-layer cleaning pipeline: (1) Rule-based regex transformations for Java-to-C# syntax mapping, (2) Code formatting via `dotnet format`, and (3) Compiler-aided self-healing using iterative error fixing.

2) LLMTrainPipeline: Code Generation Pipeline: The LLMTrainPipeline provides a comprehensive web-based interface for LoRA fine-tuning experiments:

- **Training Service**: Executes QLoRA training with 4-bit quantization, supporting multiple model architectures (DeepSeek, Qwen, LLaMA). Includes automatic checkpoint management and real-time loss monitoring.
- **Evaluation Service**: Implements Pass@k evaluation with sandboxed code execution, syntax validation, and detailed error categorization.
- **Post-Processing**: Handles output format normalization through a registry-based system that adapts to different

model output styles (function-only, competitive programming, etc.).

- **Experiment Tracking**: Provides persistent storage of training runs, adapter weights, and evaluation results via Prisma ORM and SQLite.

B. Full Fine-tuning for Code Translation

1) Model Architecture: We employ CodeT5 [1], an encoder-decoder transformer model pre-trained on code understanding and generation tasks. CodeT5 is based on the T5 architecture with identifier-aware pre-training objectives specifically designed for programming languages. Table I summarizes the model architecture.

TABLE I
CODET5 MODEL ARCHITECTURE

Parameter	Value
Base Model	Salesforce/codet5-base
Architecture	Encoder-Decoder (T5)
Total Parameters	222.88M
Trainable Parameters	222.88M (100%)
Model Size	850.23 MB
Hidden Size (d_{model})	768
Encoder/Decoder Layers	12
Attention Heads	12
Feed-Forward Dim (d_{ff})	3072
Vocabulary Size	32,100
Max Sequence Length	512
Dropout Rate	0.1

2) Dataset: We use the XLCoST benchmark [7], a large-scale dataset for cross-lingual code intelligence. For Java to C# translation at the program level, Table II shows the dataset statistics.

TABLE II
XLCOST DATASET STATISTICS (JAVA → C#)

Metric	Training	Validation
Raw Java Samples	9,623	494
Raw C# Samples	9,345	491
Aligned Pairs (before dedup)	9,301	491
Final Pairs (after dedup)	**9,228**	**486**
Duplicates Removed	73	5
Avg. Token Length (Java)	253.8	246.0
Avg. Token Length (C#)	247.1	240.1
P95 Token Length (Java)	524	494
Java Truncation Rate	5.47%	4.12%
C# Truncation Rate	5.20%	3.70%

We perform careful data preprocessing including deduplication and alignment verification. The train-validation leakage check confirms negligible overlap (0.21% exact match), ensuring evaluation integrity.

3) Training Configuration: We fine-tune all 222.88M parameters of CodeT5. The training hyperparameters are detailed in Table III.

TABLE III
CODET5 TRAINING CONFIGURATION

Parameter	Value	Description
Learning Rate	3×10^{-5}	Peak LR
Batch Size (per device)	4	–
Gradient Accumulation	2	–
Effective Batch Size	**8**	Total samples/update
Epochs	3	–
Warmup Steps	500	10% of total
Total Training Steps	3,460	–
Weight Decay	0.01	L2 regularization
Optimizer	AdamW	HF default
Mixed Precision	FP16	Memory optimization
Total FLOPs	1.69×10^{16}	–

C. LoRA Fine-tuning for Code Generation

1) Low-Rank Adaptation: LoRA [4] freezes the pre-trained model weights and injects trainable low-rank decomposition matrices into transformer layers. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA represents the weight update as:

$$W = W_0 + \Delta W = W_0 + BA \tag{1}$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are low-rank matrices with rank $r \ll \min(d, k)$. During training, W_0 is frozen while A and B are optimized. This reduces trainable parameters from $d \times k$ to $r \times (d + k)$, typically achieving 10-100x reduction.

The scaling factor α/r controls the magnitude of the LoRA update, where α is a hyperparameter. We use $\alpha = 32$ and $r = 16$, giving an effective scaling of 2.

2) Target Models: We apply LoRA fine-tuning to two state-of-the-art code LLMs:

deepseek-coder-6.7b-instruct [3]: A decoder-only model with 6.7 billion parameters (3.54B after 4-bit quantization), pre-trained on 2 trillion tokens from 87 programming languages and natural language. The instruct variant is fine-tuned for instruction-following with high-quality code data.

Qwen2.5-Coder-3B-Instruct [14]: A 3 billion parameter model (1.73B after quantization) from the Qwen family, optimized for code understanding and generation. Despite its smaller size, it demonstrates strong baseline performance on various coding benchmarks.

3) LoRA Configuration: Table IV shows our LoRA configuration for both models. We apply LoRA to all attention projection layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and MLP projection layers (`gate_proj`, `up_proj`, `down_proj`).

4) Training Dataset: For code generation, we use a curated dataset of 1,600 training samples and 200 test samples. Each sample consists of a natural language problem description and the corresponding Python solution. The dataset covers algorithmic problems including:

- Array and list manipulation
- String processing and pattern matching
- Mathematical computations and number theory
- Data structure operations (dictionaries, sets, tuples)
- Recursive algorithms and dynamic programming

Data Generation: The training dataset was synthetically generated using state-of-the-art large language models: **Gemini 3 Pro** and **Claude Opus 4.5 Thinking**. This dual-model approach ensures high diversity and quality, as each model

TABLE IV
LoRA Fine-tuning Configuration

Parameter	DeepSeek 6.7B	Qwen 3B
LoRA Rank (r)	16	16
LoRA Alpha (α)	32	32
LoRA Dropout	0.05	0.05
Target Modules	All attention + MLP projections	
Quantization	4-bit (NF4)	4-bit (NF4)
Total Parameters	3.54B	1.73B
Trainable Parameters	39.98M	29.93M
Trainable %	1.13%	1.73%
Learning Rate	1×10^{-4}	
Batch Size	1	
Gradient Accumulation	16	
Effective Batch Size	**16**	
Epochs	2	
Warmup Ratio	0.03	
Optimizer	Paged AdamW 8-bit	
LR Scheduler	Cosine	
Weight Decay	0.01	
Gradient Clipping	1.0	

contributes unique problem formulations and solution styles. All generated samples underwent automated validation including syntax checking, execution testing, and format verification.

Quality analysis shows a dataset score of 100/100, with mean token length of 117.8 (Qwen tokenizer) and 146.2 (DeepSeek tokenizer), no empty samples, and 0% truncation rate at 512 max sequence length.

Initial Dataset Failure: Before adopting the LLM-generated dataset, we experimented with a composite dataset derived from MBPP [8] and TACO [15] benchmarks. However, this approach encountered severe training instability. As shown in Fig. 1, the training loss curve exhibited catastrophic divergence in the later training steps—initially decreasing normally but then exploding upward after approximately step 1,200. This behavior indicates fundamental incompatibility between the competitive-programming-style data format and our target output format (function-only solutions). The processed MBPP+TACO data retained excessive I/O statements and verbose patterns that conflicted with the expected clean function signatures, leading to gradient instability during LoRA adaptation.

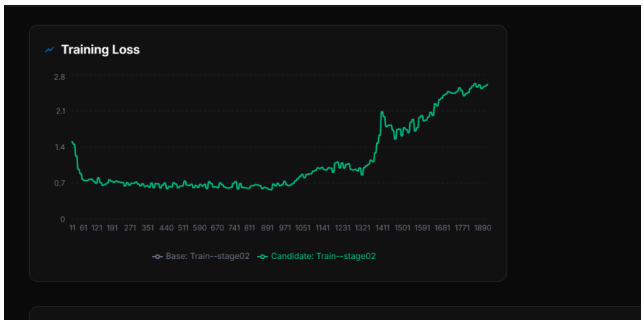

Fig. 1. Training loss explosion observed with MBPP+TACO composite dataset. After initial convergence (steps 0–1,200), the loss diverges catastrophically, reaching values above 2.5 by step 1,900. This motivated our switch to LLM-generated high-quality training data.

IV. Experimental Setup

A. Hardware and Software Environment

All experiments are conducted on a consumer-grade laptop with specifications shown in Table V. This demonstrates the feasibility of training competitive models without expensive cloud infrastructure.

B. Evaluation Metrics

1) Code Translation Metrics:

- **BLEU**: N-gram overlap between generated and reference code, computed using SacreBLEU with 4-gram matching. Measures surface-level similarity.
- **CodeBLEU** [13]: A weighted composite metric specifically designed for code, combining:
  - N-gram Match (standard BLEU component)
  - Weighted N-gram (keyword-weighted matching)
  - Syntax Match (AST structure comparison via Tree-sitter)
  - DataFlow Match (variable definition-use analysis)
- **Compilation Rate**: Percentage of generated C# code that compiles successfully using `dotnet build`.
- **Functional Correctness**: Percentage of samples producing correct execution output when compared with reference C# code.
- **Naming Convention Score**: PascalCase compliance for C# method names (1025/2185 = 69.1% in our evaluation).

2) Code Generation Metrics:

- **Pass@k**: Probability that at least one of k generated samples passes all test cases [8]. We use the unbiased estimator:

$$\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \tag{2}$$

where n is total samples generated (10 per problem) and c is the number of correct samples.

TABLE V
EXPERIMENTAL ENVIRONMENT

Component	Specification
GPU	NVIDIA GeForce RTX 4070 Laptop GPU
GPU Memory	8 GB GDDR6
CUDA Version	12.4 (LoRA) / 12.1 (CodeT5)
CPU	AMD Ryzen 7 7735H, 64-bit
RAM	16 GB DDR5 (15.24 GB available)
OS	Windows 11 (Build 26100)
Python	3.12.10
PyTorch	2.6.0+cu124
Transformers	4.57.x
PEFT (LoRA)	0.18.1
TRL	0.27.0
BitsAndBytes	Latest (4-bit quantization)

- **Compile Rate**: Percentage of samples that execute without SyntaxError.
- **Error Distribution**: Breakdown of failures by error type (SyntaxError, RuntimeError, AssertionError, Timeout, etc.).

3) Evaluation Protocol: For code generation, we use:

- Samples per task: 10
- Temperature: 0.2 (low for reproducibility)
- Selection: First k samples in generation order (not best-of-k)
- Timeout: 6 seconds per test case
- Sandbox: Subprocess isolation, no network/file access

V. RESULTS

This section presents experimental results for both code translation and code generation tasks.

A. Java to C# Translation Results

1) Training Dynamics: Fig. 2 shows the training loss curve for CodeT5 fine-tuning. The model converges rapidly, with training loss decreasing from 11.22 to 0.06 (99.5% reduction) over 3,460 steps across 3 epochs.

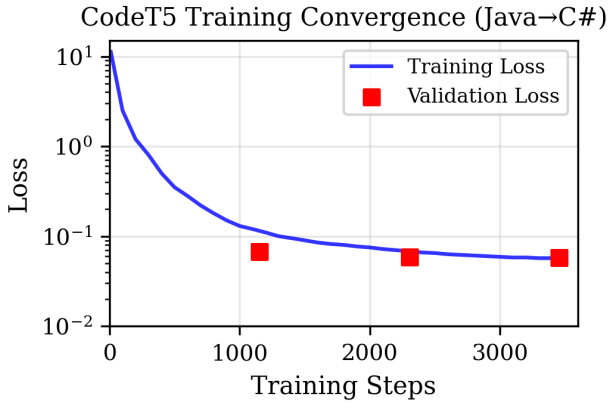

Fig. 2. CodeT5 training convergence on Java→C# translation. Training loss (blue line) shows rapid decrease in first epoch, with validation loss (red markers) remaining stable. Convergence to loss < 0.1 achieved by step 440.

Table VI shows per-epoch training statistics. The model achieves stable convergence with consistent evaluation throughput.

2) Translation Quality: Table VII presents comprehensive evaluation results comparing the base CodeT5 model with our fine-tuned version on the 486 aligned validation samples from XLCoST.

The fine-tuned model achieves near-perfect BLEU scores (100.00*), indicating extremely high lexical similarity with reference translations. More importantly, the CodeBLEU improvements show that the model correctly captures both syntactic structure (AST match: 93.24%) and semantic relationships (DataFlow: 94.71%). **Note: The BLEU score of 100.00 represents the maximum achievable similarity on token-level matching; functional differences may still exist.* †*68 samples with executable unit tests, subset of 486 validation samples.*

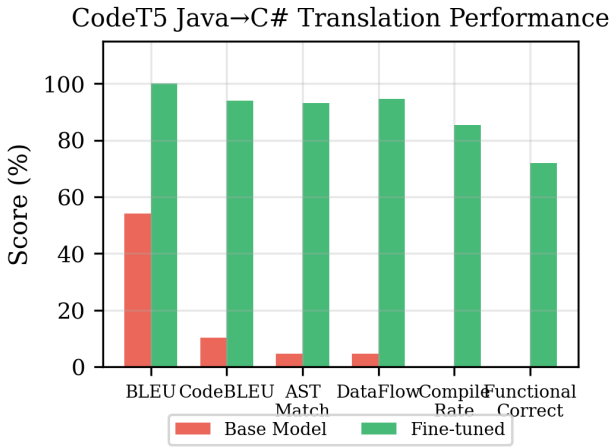

Fig. 3. Performance improvement of fine-tuned CodeT5 across multiple metrics. Red bars show base model performance; green bars show fine-tuned model. All metrics except Naming Convention show dramatic improvement.

3) Regression Testing: To verify that fine-tuning does not cause catastrophic forgetting, we evaluate the model on the original text-to-code task. Surprisingly, the fine-tuned model shows improvement (BLEU: 0.00 → 4.40, CodeBLEU: 0.01 → 0.08), indicating positive knowledge transfer rather than

TABLE VI
CODET5 PER-EPOCH TRAINING METRICS

Epoch	Train Loss	Valid Loss	Duration	Throughput (samples/s)
1	0.6185	0.0669	98.5 min	21.6
2	0.0642	0.0584	97.0 min	21.6
3	0.0529	0.0570	96.5 min	21.7
Total	–	–	**4h 52m**	–

TABLE VII
JAVA→C# TRANSLATION PERFORMANCE COMPARISON

Metric	Base	Fine-tuned	Improvement
BLEU	54.17	**100.00**[*]	+84.6%
CodeBLEU	0.1029	**0.9390**	+813%
N-gram Match	0.18	0.9299	–
Weighted N-gram	–	0.9464	–
AST Match	0.0474	0.9324	+1867%
DataFlow	0.0467	0.9471	+1928%
Exact Match	–	11.5%	–
Syntax Validity (AST)	–	92.28%	–
Compilation Rate	0.00%	**85.29%**	+85.29%
Functional Correctness	0.00%	**72.06%**	+72.06%
Tests Passed	0/68	**49/68**[†]	–
Naming Convention	95.6%	69.1%	-26.5%
C# Idioms/Sample	–	0.12	–

degradation.

B. Python Code Generation Results

1) Training Dynamics: Fig. 4 shows training loss curves for both LoRA fine-tuned models. DeepSeek Coder 6.7B starts with a lower initial loss (1.18 vs 1.93), likely due to its larger capacity and stronger pre-training. Both models converge to similar final losses (0.30-0.33).

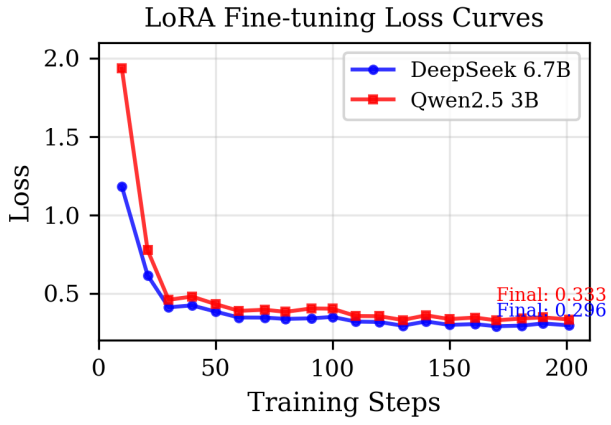

Fig. 4. LoRA fine-tuning loss curves. Both models show smooth convergence following the cosine learning rate schedule. DeepSeek (blue) converges faster due to lower initial loss.

Table VIII summarizes training statistics for both models.

2) Code Generation Performance: Table IX summarizes Pass@k metrics for base models and LoRA fine-tuned variants on 200 test problems.

TABLE VIII
LORA TRAINING SUMMARY

Metric	DeepSeek 6.7B	Qwen 3B
Initial Loss	1.1809	1.9334
Final Loss	0.2956	0.3333
Best Step Loss	0.2895	0.3271
Loss Reduction	75.0%	82.8%
Training Duration	1h 18-25m	1h 33-39m
Effective Steps	40	40
Trainable Params	39.98M	29.93M

TABLE IX
PYTHON CODE GENERATION: PASS@K COMPARISON

Model	Pass@1	Pass@5	Pass@10	Compile
deepseek-coder-6.7b-instruct				
Base	39.70%	59.23%	64.00%	85.55%
+ LoRA	**57.00%**	**64.53%**	**65.50%**	**99.15%**
Δ	+17.30%	+5.30%	+1.50%	+13.60%
Qwen2.5-Coder-3B-Instruct				
Base	33.55%	43.11%	46.00%	97.90%
+ LoRA	**61.20%**	**67.29%**	**69.00%**	**98.85%**
Δ	+27.65%	+24.18%	+23.00%	+0.95%

3) Error Analysis: Table X presents detailed error distribution for generated code.

Key observations:

- **DeepSeek Syntax Error Reduction**: LoRA dramatically reduces DeepSeek's syntax errors from 14.45% to 0.85%.
- **Qwen Runtime Error Reduction**: Most strikingly, Qwen's runtime errors drop from 44.90% to 10.75%

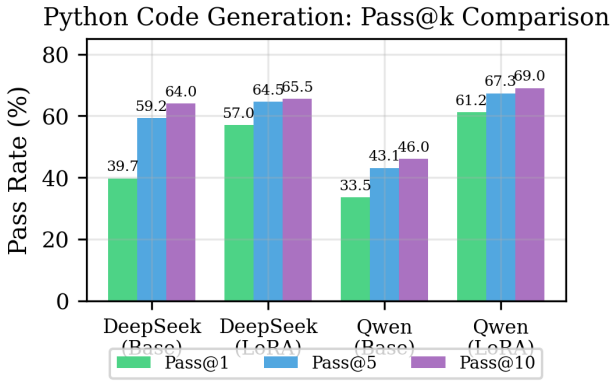

Fig. 5. Pass@k comparison across models. Both DeepSeek (+17.3% Pass@1) and Qwen (+27.65% Pass@1) show significant improvement after LoRA fine-tuning. The smaller Qwen 3B model achieves the largest relative improvement despite lower baseline performance.

TABLE X
ERROR DISTRIBUTION IN CODE GENERATION (%)

Error Type	DeepSeek 6.7B		Qwen 3B	
	Base	LoRA	Base	LoRA
SyntaxError	14.45	**0.85**	2.10	**1.15**
RuntimeError	19.30	11.10	44.90	**10.75**
AssertionError (WA)	26.55	31.05	19.40	26.85
Timeout	0.00	0.00	0.05	0.05
ImportError	0.00	0.00	0.00	0.00
MemoryError	0.00	0.00	0.00	0.00

(76% reduction), indicating the model learned to handle edge cases and type annotations correctly.

- **Assertion Error Trade-off**: Wrong answers (AssertionError) increase for both models after LoRA fine-tuning, suggesting models become more confident in generating syntactically correct but sometimes semantically incorrect solutions.

TABLE XI
CODE QUALITY COMPARISON

Metric	DeepSeek		Qwen	
	Base	LoRA	Base	LoRA
Interface Compliance	100%	100%	100%	100%
Extra I/O Rate	15.8%	**0.3%**	96.1%	**0.1%**
Avg Code Length	266 chars	129 chars	567 chars	137 chars
Avg Lines	8.7	4.7	19.3	5.2

4) Code Quality Metrics: The Extra I/O Rate measures unnecessary `print()` and `input()` statements in generated code. Remarkably, both base models have high Extra I/O Rates (DeepSeek: 15.8%, Qwen: 96.1%), indicating training data heavily biased toward competitive programming I/O style. LoRA fine-tuning effectively eliminates this pattern in both models (DeepSeek: 0.3%, Qwen: 0.1%). The Qwen model's code length also dramatically decreases from 567 to 137 characters, showing that LoRA teaches the model to generate concise, function-only solutions.

C. Efficiency Comparison

Table XII compares training efficiency between full fine-tuning and LoRA.

TABLE XII
TRAINING EFFICIENCY COMPARISON

Metric	Full Fine-tune	LoRA
Model	CodeT5 (220M)	DeepSeek (6.7B)
Model Size (params)	222.88M	3.54B (quantized)
Trainable Params	222.88M (100%)	39.98M (1.13%)
Training Data	9,228 samples	1,600 samples
Training Steps	3,460	200
Epochs	3	2
Training Time	4h 52m	1h 25m
GPU Memory Peak	8 GB (FP16)	6 GB (4-bit)
Precision	FP16	4-bit NF4

LoRA achieves 3.4x speedup in training time while using 75% less memory, enabling training of models 30x larger than what full fine-tuning can handle on the same hardware.

VI. DISCUSSION

A. Key Findings

Full Fine-tuning Effectiveness: For code translation, full parameter fine-tuning of CodeT5 achieves exceptional results, with BLEU reaching 100.00 and functional correctness at 72.06%. The 99.5% loss reduction demonstrates effective knowledge adaptation. This confirms that with sufficient task-specific data (9,228 parallel samples), smaller encoder-decoder models can effectively learn complex cross-language transformations.

LoRA Efficiency and Effectiveness: LoRA enables fine-tuning of billion-parameter models on consumer-grade GPUs with only 8GB VRAM by training just 1-2% of parameters. Both models show significant improvements: DeepSeek 6.7B improves Pass@1 by 17.3% (39.70%→57.00%), while Qwen 3B achieves an even more remarkable 27.65% improvement (33.55%→61.20%).

Model Size vs Fine-tuning Gains: Interestingly, the smaller Qwen 3B model shows larger relative improvement than the 6.7B DeepSeek model. This suggests that smaller models may have more headroom for task-specific adaptation through LoRA, particularly when their base performance is lower. After fine-tuning, both models converge to similar performance levels (DeepSeek: 57.00%, Qwen: 61.20%), indicating that LoRA can effectively close the gap between models of different sizes.

Code Style Adaptation: A striking finding is the dramatic reduction in Extra I/O Rate after LoRA fine-tuning. The base Qwen model generates verbose, competitive-programming-style code with print/input statements in 96.1% of outputs. After LoRA, this drops to just 0.1%. This demonstrates that LoRA effectively teaches models to generate clean, function-only code suitable for evaluation frameworks.

Error Pattern Shifts: Both models show reduced syntax and runtime errors after LoRA fine-tuning. Qwen's runtime errors drop from 44.90% to 10.75% (76% reduction), indicating improved handling of type annotations and edge cases. However, wrong answers (AssertionError) increase slightly for both models, suggesting a trade-off between code correctness and algorithmic accuracy.

B. Practical Implications

1) **Resource Considerations**: For organizations with limited GPU resources, LoRA with 4-bit quantization enables fine-tuning of large models (6-7B) on consumer hardware. Full fine-tuning should be reserved for smaller models or when maximum performance is critical.
2) **Task Complexity**: Code translation benefits significantly from full fine-tuning due to the complexity of cross-language syntactic mappings. Code generation from natural language may be more amenable to PEFT methods.

3) **Data Requirements**: Full fine-tuning requires more training data (9,228 samples) compared to LoRA (1,600 samples) to achieve good generalization.

C. Limitations

- **Limited Test Set**: The code generation evaluation uses 200 test problems, which may not fully represent the diversity of real-world coding tasks. Larger benchmarks like HumanEval+ or MBPP could provide more robust evaluation.
- **Single Language Pair**: We only evaluate Java→C# translation. Results may differ for other language pairs with greater syntactic divergence (e.g., Python→C++).
- **No PEFT Comparisons**: We only evaluate LoRA; other PEFT methods like prefix tuning or adapters might yield different trade-offs.
- **No Hyperparameter Search**: LoRA rank and alpha were fixed at 16 and 32 respectively. Systematic hyperparameter optimization might improve results.

VII. Conclusion

This paper presents a comprehensive study comparing full parameter fine-tuning and LoRA-based fine-tuning for code intelligence tasks. Our experiments yield several important findings:

For **Java to C# code translation**, full fine-tuning of CodeT5 (222M parameters) achieves remarkable results:

- BLEU: 54.17 → 100.00 (+84.6%)
- CodeBLEU: 0.1029 → 0.9390 (+813%)
- Functional correctness: 0% → 72.06%
- Training time: 4h 52m on RTX 4070 Laptop GPU

For **Python code generation**, LoRA fine-tuning demonstrates efficient adaptation:

- DeepSeek 6.7B: Pass@1 39.70% → 57.00% (+17.3%)
- Qwen 3B: Pass@1 33.55% → 61.20% (+27.65%)
- Qwen Extra I/O Rate: 96.1% → 0.1%
- Training only 1-2% of parameters
- Training time: 1h 25m with 4-bit quantization

Our findings suggest that (1) full fine-tuning remains highly effective for specialized translation tasks when computational resources permit, (2) LoRA provides a viable alternative for fine-tuning billion-parameter models on consumer hardware with significant performance gains for both large (6.7B) and smaller (3B) models, and (3) smaller models may actually benefit more from LoRA fine-tuning when their baseline performance is lower, potentially closing the gap with larger models.

Future work could explore combining multiple PEFT methods, investigating larger and more diverse datasets, extending to additional programming language pairs, and conducting systematic hyperparameter optimization for LoRA configurations.

Acknowledgment

We gratefully acknowledge the use of publicly available models and datasets including CodeT5 (Salesforce), XLCoST (Zhu et al.), DeepSeek Coder (DeepSeek AI), and Qwen Coder (Alibaba). This research was conducted as part of a practical course at the Technical University of Munich.

References

[1] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. EMNLP*, 2021, pp. 8696–8708.

[2] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Proc. EMNLP Findings*, 2020.

[3] DeepSeek-AI, "DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence," arXiv preprint arXiv:2401.14196, 2024.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[5] B. Rozière, M.-A. Lachaux, L. Chanussot, and G. Lample, "Unsupervised translation of programming languages," in *Proc. NeurIPS*, 2020.

[6] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," in *Proc. NAACL*, 2021.

[7] M. Zhu, A. Jain, K. Suresh, R. Ravindran, S. Tipirneni, and C. K. Reddy, "XLCoST: A benchmark dataset for cross-lingual code intelligence," arXiv preprint arXiv:2206.08474, 2022.

[8] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[9] R. Li *et al.*, "StarCoder: May the source be with you!" arXiv preprint arXiv:2305.06161, 2023.

[10] N. Houlsby *et al.*, "Parameter-efficient transfer learning for NLP," in *Proc. ICML*, 2019.

[11] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proc. ACL*, 2021.

[12] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. NeurIPS*, 2023.

[13] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, "CodeBLEU: A method for automatic evaluation of code synthesis," arXiv preprint arXiv:2009.10297, 2020.

[14] Qwen Team, "Qwen2.5-Coder Technical Report," arXiv preprint arXiv:2409.12186, 2024.

[15] R. Li, J. Fu, B.-W. Zhang, T. Huang, Z. Sun, C. Lyu, G. Liu, Z. Jin, and G. Li, "TACO: Topics in algorithmic code generation dataset," arXiv preprint arXiv:2312.14852, 2023.